

TP Final : Analyse Complète d'un Ransomware

Contexte Pédagogique

Ce TP final combine toutes les techniques apprises dans les TPs précédents :

- **TP1** : Analyse statique avec dnSpy et désobfuscation
- **TP2** : Analyse dynamique avec strace, tcpdump, et détection réseau

Objectif : Analyser un ransomware sophistiqué utilisant plusieurs techniques d'obfuscation, de persistance, et de chiffrement.

Préparation de l'Environnement

1. Installation des Dépendances

```
# Installer les outils nécessaires
pip3 install cryptography

# Installer les outils de monitoring (si pas déjà installés)
sudo apt-get install -y inotify-tools

# Vérifier les outils système
which strace tcpdump inotifywait
```

2. Structure du Laboratoire

```
RansomwareRE/
├── sample/
│   └── cryptolocker_advanced.py # Échantillon à analyser
└── README.md                  # Ce fichier
```

3. Préparation de l'Échantillon

```
cd /root/Cours/RansomwareRE/sample
chmod +x cryptolocker_advanced.py

# NE PAS EXÉCUTER IMMÉDIATEMENT !
# Toujours commencer par l'analyse statique
```

Phase 1 : Reconnaissance Initiale

Objectifs

- Identifier le type de malware
- Calculer les hashes pour identification
- Détecter les techniques d'obfuscation
- Identifier les imports suspects

1.1 Calcul des Hashs

```
# Calculer MD5, SHA1, SHA256
md5sum cryptolocker_advanced.py
sha1sum cryptolocker_advanced.py
sha256sum cryptolocker_advanced.py
```

Questions :

- Q1: Quels sont les trois hashes de l'échantillon ?
- Q2: Comment ces hashes peuvent-ils être utilisés pour la détection ?

1.2 Analyse des Métadonnées

```
# Informations basiques
file cryptolocker_advanced.py
wc -l cryptolocker_advanced.py
stat cryptolocker_advanced.py
```

Questions :

- Q3: Quel est le type de fichier ?
- Q4: Combien de lignes contient le code ?

1.3 Analyse des Imports

```
# Extraire les imports Python
grep -E "^import|^from " cryptolocker_advanced.py
```

Questions :

- Q5: Quels imports suggèrent des capacités de chiffrement ?
- Q6: Quels imports indiquent des communications réseau ?
- Q7: Quels imports sont suspects pour un ransomware ?

Phase 2 : Analyse Statique Approfondie

Objectifs

- Désobfusquer les chaînes encodées
- Identifier la configuration C2
- Comprendre l'algorithme de chiffrement
- Extraire les IOCs statiques

2.1 Extraction des Chaînes

```
# Extraire toutes les chaînes lisibles
strings cryptolocker_advanced.py | less

# Rechercher des patterns spécifiques
grep -i "bitcoin\\|btc\\|ransom\\|decrypt\\|encrypt" cryptolocker_advanced.py
```

Questions :

- Q8: Quelle est l'adresse Bitcoin demandée ?
- Q9: Quel est le montant de la rançon ?
- Q10: Quel est le délai imposé aux victimes ?

2.2 Désobfuscation des Variables

Vous remarquerez des variables obfusquées comme `_0x1a2b`, `_0x3c4d`, etc.

```
# Rechercher les assignments de variables obfusquées
grep "_0x" cryptolocker_advanced.py
```

Questions :

- Q11: Que contient la variable `_0x1a2b` ?
- Q12: Que contient la variable `_0x3c4d` ?
- Q13: À quoi sert `_0x5e6f` ?

Exercice de Désobfuscation : Créez un script Python pour décoder automatiquement la base 64.

```
import base64
```

2.3 Analyse des Méthodes

Le code utilise des noms de méthodes obfusqués : `_0x1()`, `_0x2()`, etc.

Questions :

- Q14: Que fait la méthode `_0x1()` ?
- Q15: Que fait la méthode `_0x2()` ?
- Q16: Que fait la méthode `_0x3()` ?
- Q17: Que fait la méthode `_0x4()` ?

- Q17bis: Que fait la méthode `_0x4bis()` ?
- Q18: Que fait la méthode `_0x5()` ?
- Q19: Que fait la méthode `_0x6()` ?
- Q20: Que fait la méthode `_0x7()` ?
- Q21: Que fait la méthode `_0x8()` ?
- Q22: Que fait la méthode `_0x9()` ?
- Q2E: Que fait la méthode `_0x10()` ?

2.4 Analyse de l'Algorithme de Chiffrement

```
# Rechercher les fonctions cryptographiques
grep -i "fernet\|aes\|cipher\|encrypt\|kdf\|pbkdf2"
cryptolocker_advanced.py
```

Questions :

- Q23: Quel algorithme de chiffrement est utilisé ?
- Q24: Comment la clé de chiffrement est-elle dérivée ?
- Q25: Combien d'itérations sont utilisées pour PBKDF2 ?
- Q26: Quel est le sel utilisé pour la dérivation de clé ?

Phase 3 : Analyse Dynamique

Objectifs

- Observer le comportement en temps réel
- Capturer les appels système
- Monitorer le trafic réseau
- Identifier les fichiers créés/modifiés

3.1 Préparation de l'Environnement de Test

```
# Créer un répertoire de test isolé
mkdir -p ~/Documents/VictimFiles

# Créer des fichiers de test
for i in {1..5}; do
    echo "Important document $i" > ~/Documents/VictimFiles/doc_$i.txt
    echo "Photo data $i" > ~/Documents/VictimFiles/photo_$i.jpg
done

# Vérifier
ls -la ~/Documents/VictimFiles/
```

3.2 Traçage des Appels Système

```
# Lancer strace avec filtres complets
strace -f -e trace=openat,unlink,socket,connect,write -o strace_ransom.log \
    python3 cryptolocker_advanced.py 2>&1 | tee execution.log
```

Questions :

- Q27: Quels fichiers sont ouverts en lecture ? (cherchez `openat` avec `O_RDONLY`)
- Q28: Quels fichiers sont supprimés ? (cherchez `unlink`)
- Q29: Quels fichiers sont créés ? (cherchez `openat` avec `O_CREAT`)
- Q30: Y a-t-il une tentative de connexion réseau ?

Commandes d'Analyse :

```
# Fichiers ouverts
grep "openat.*O_RDONLY" strace_ransom.log

# Fichiers supprimés
grep "unlink" strace_ransom.log

# Connexions réseau
grep "?" strace_ransom.log
```

3.3 Surveillance en Temps Réel des Fichiers

Terminal 1 (Monitoring) :

```
apt-get update && apt-get install -y inotify-tools # If not installed

inotifywait -m -r ~/Documents/VictimFiles/ \
    -e create,delete,modify,moved_to,moved_from \
    2>&1 | tee inotify_ransom.log
```

Terminal 2 (Exécution) :

```
python3 cryptolocker_advanced.py
```

Questions :

- Q31: Quels événements sont détectés par inotify ?
- Q32: Dans quel ordre les fichiers sont-ils modifiés ?
- Q33: Quelle extension est ajoutée aux fichiers chiffrés ?

3.4 Capture du Trafic Réseau

```
# Lancer la capture en arrière-plan
sudo tcpdump -i any -l -A -s 0 \
    'host 192.168.1.100 and port 8080' \
    2>&1 | tee tcpdump_ransom.log &

# Obtenir le PID
TCPDUMP_PID=$!

# Exécuter le ransomware
python3 cryptolocker_advanced.py

# Attendre quelques secondes
sleep 3

# Arrêter la capture
sudo kill $TCPDUMP_PID
```

Questions :

- Q34: Quelle adresse IP le ransomware tente-t-il de contacter ?
- Q35: Sur quel port ?
- Q36: Quelles données sont envoyées au serveur C2 ?
- Q37: Y a-t-il une réponse du serveur ?

Phase 4 : Extraction des IOCs et Documentation

Objectifs

- Compiler tous les Indicateurs de Compromission
- Créer des règles de détection
- Proposer des mesures de remédiation

4.1 Checklist des IOCs

Complétez cette liste avec vos découvertes :

IOCs Réseau

- ☐ Adresse IP C2 : _____
- ☐ Port C2 : _____
- ☐ Protocole : _____
- ☐ Format des données envoyées : _____

IOCs Fichiers

- ☐ Mutex : _____

- ☐ Chemin d'installation : _____
- ☐ Nom du fichier de persistance : _____
- ☐ Extension des fichiers chiffrés : _____
- ☐ Nom de la note de rançon : _____
- ☐ Fichiers temporaires créés : _____

IOCs Cryptographiques

- ☐ Algorithme de chiffrement : _____
- ☐ Algorithme de dérivation de clé : _____
- ☐ Sel utilisé : _____
- ☐ Nombre d'itérations PBKDF2 : _____

IOCs Comportementaux

- ☐ Extensions de fichiers ciblées : _____
- ☐ Limite de fichiers chiffrés : _____
- ☐ Commandes de suppression de backups : _____

IOCs Financiers

- ☐ Adresse Bitcoin : _____
- ☐ Montant demandé : _____
- ☐ Email de contact : _____
- ☐ Délai de paiement : _____

4.2 Création de Règles YARA

Créez une règle YARA pour détecter ce ransomware :

```
rule CryptoLocker_Educational {
  meta:
    description = "Détection le ransomware CryptoLocker éducatif"
    author = "Votre Nom"
    date = "2024"

  strings:
    // À compléter avec vos découvertes
    $bitcoin = "1BoatSLRHtKNgkdXEeobR76b53LETtpyT"
    $mutex = "_____ "
    $c2_encoded = "_____ "
    $ransom_note = "_____ "
    $salt = "_____ "

  condition:
    // Définir votre condition
    _____
}
```

4.3 Règle Snort

Créez une règle Snort pour détecter la communication C2 :

```
alert tcp $HOME_NET any -> _____ (
  msg:"CryptoLocker C2 Communication Detected";
  content:"VICTIM:";
  content:"|_____|";
  sid:1000002;
  rev:1;
)
```

Phase 5 : Remédiation et Prévention (Bonus)

5.1 Supprimer la Persistance

```
# Identifier les mécanismes de persistance
ls -la ~/.config/autostart/

# Supprimer
rm ~/.config/autostart/WindowsDefenderUpdate.desktop

# Supprimer l'installation
rm -rf ~/.system/WinDefender/
```

5.2 Tentative de Déchiffrement

Question Avancée :

- Q38: Est-il possible de déchiffrer les fichiers sans payer la rançon ?
- Q39: Quelles informations sont nécessaires pour déchiffrer ?
- Q40: Où pourrait-on trouver la clé de déchiffrement ?

Indice : Analysez comment la clé est générée dans `_0x?()`.

5.3 Script de Déchiffrement (Challenge)

Créez un script Python pour déchiffrer les fichiers si vous avez le victim ID :

```
#!/usr/bin/env python3
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2
import base64
import sys

def derive_key(victim_id):
```



```
_salt = b'<SALTED_KEY>'
_kdf = PBKDF2(
    algorithm=hashes.SHA256(),
    length=32,
    salt=_salt,
    iterations=100000,
)
return base64.urlsafe_b64encode(_kdf.derive(victim_id.encode()))

def decrypt_file(encrypted_path, key):
    """
    Pseudo-code pour déchiffrer un fichier :

    1. Lire le contenu du fichier chiffré (.locked)
    2. Créer un objet Fernet avec la clé
    3. Déchiffrer les données avec fernet.decrypt()
    4. Extraire le chemin du fichier original (enlever .locked)
    5. Écrire les données déchiffrées dans le fichier original
    6. Supprimer le fichier .locked
    7. Retourner True si succès, False sinon
    """
    pass

if __name__ == "__main__":
    if len(sys.argv) != 2:
        print("Usage: decrypt.py <VICTIM_ID>")
        sys.exit(1)

    victim_id = sys.argv[1]
    key = derive_key(victim_id)

    # Déchiffrer tous les fichiers .locked
    """
    Pseudo-code pour la boucle principale :

    1. Scanner le répertoire ~/Documents/VictimFiles/
    2. Trouver tous les fichiers avec extension .locked
    3. Pour chaque fichier .locked trouvé :
        - Appeler decrypt_file(chemin, key)
        - Afficher un message de succès ou d'erreur
    4. Afficher un résumé : nombre de fichiers déchiffrés
    """
```

Ressources Complémentaires

Documentation Python Cryptography

- <https://cryptography.io/en/latest/fernet/>
- <https://cryptography.io/en/latest/hazmat/primitives/key-derivation-functions/>

Outils d'Analyse

- **strace** : Traçage des appels système
- **tcpdump** : Capture de paquets réseau
- **inotifywait** : Surveillance de fichiers en temps réel
- **strings** : Extraction de chaînes lisibles
- YARA : Création de signatures de détection

Recherche de Ransomware

- No More Ransom Project : <https://www.nomoreransom.org/>
- ID Ransomware : <https://id-ransomware.malwarehunterteam.com/>

Notes de Sécurité

⚠ IMPORTANT :

- Ce ransomware est limité à 15 fichiers pour éviter les dommages
- Il ne chiffre que dans `~/Documents/VictimFiles/`
- Il ne contacte PAS de vrai serveur C2 (IP fictive)
- Toujours travailler dans une VM isolée

Bon travail ! 🔒🔍